CMU 11-768 Homework 2 Report

Validating and Packaging Agent Visualization Tasks

CMU 11-768 Homework 2 Report: Validating and Packaging Agent Visualization Tasks

1. Part 1 — Hybrid Validator Design (`validator/solution.py`)

1.1 Limitations of the Provided Baseline (`validator/baseline.py`)
The reference baseline in `validator/baseline.py` issues four generic VLM calls (`complete()`) per run—one for each `ErrorFamily` (`execution_failure`, `wrong_data`, `wrong_chart`, `hard_to_read`)—passing the raw stringified `run.messages` list, raw input files, and the rendered PNG. Inspecting the 30 seed runs in `artifacts/001`–`030` reveals three recurring failure modes in the baseline:
1. **Wasted VLM calls and hallucinations on missing/corrupt artifacts**: When `run.figure` is `None` (e.g., `artifacts/001`, where the agent saved to `plots/box_plot.png` instead of `figure.png`), the baseline still invokes the VLM without an image and relies on the model noticing that no image was attached.
2. **Conflation of `wrong_data` and `wrong_chart`**: Generic prompts ("Does the plotted data differ from what was requested?") cause VLMs to flag visual design issues (e.g., horizontal vs. vertical bars, missing axis labels) as `wrong_data`, or conversely miss silent data bugs (e.g., omitting a `status == 'completed'` filter or computing an unweighted mean instead of a weighted ratio) because the resulting bar heights still look plausible in the PNG.
3. **Trajectory noise**: Stringifying `run.messages` includes lengthy system prompts, bash boilerplate, and intermediate failed commands, diluting the final plotting script that actually generated `figure.png`.

1.2 Two-Stage Hybrid Validator Architecture
Our implementation in `validator/solution.py` combines fast, deterministic program/image inspection with structured VLM judgment:

1. **`judge_execution(run)`**:
   - **Deterministic file & path analysis**: Immediately checks `run.figure is None or not run.figure.is_file()`. If missing, it scans the agent's bash tool calls for `savefig(...)` arguments to report the exact wrong path (e.g., saving to `'plots/box_plot.png'` instead of `'figure.png'`).
   - **Structural PNG & blank-canvas verification**: Verifies byte size (`>= 500` bytes), PIL `img.verify()`, `img.format == "PNG"`, minimum dimensions (`>= 64x64`), distinct RGB color count (`> 2`), and background pixel dominance (`<= 99.8%` single color).
2. **`judge_data_and_chart(run)`**:
   - **Chronological command extraction (`_extract_commands_and_outputs`, `_extract_final_python_code`)**: Parses `run.messages` to pair each assistant `bash` tool call with its tool returncode and stdout/stderr, isolating the last successful Python plotting command(s).
   - **Programmatic `WRONG_DATA` checks**:
     - *Input bypass / hardcoding detection*: Flags runs where `run.inputs` provides CSV/JSON files that are never opened or referenced in the plotting code.
     - *Filter token verification*: Extracts backtick-quoted filter values from clauses containing `only`, `whose ... is`, `exclude`, or `ignore` and verifies they appear in the plotting script.
     - *Aggregation operator verification*: Detects mismatches between weighted averages vs. unweighted `.mean()`, `median` vs. `.mean()`, and missing cumulative sums (`cumsum`).
   - **Programmatic `WRONG_CHART` checks**:
     - Verifies chart family calls (`boxplot`, `barh` vs. `bar`, `scatter`, `imshow`/`heatmap`), logarithmic scale calls (`set_xscale('log')`/`set_yscale('log')`), required title and both axis labels (`set_title`, `set_xlabel`, `set_ylabel`), error bars (`yerr`/`xerr`/`errorbar`), and dashed reference lines (`axhline`/`axvline`).
   - **Structured VLM fallback (`_ask_vlm`)**: When a live VLM endpoint (`VALIDATOR_BASE_URL` / `VALIDATOR_API_KEY`) is configured, queries the VLM with compacted step-by-step trajectory summaries, input excerpts, and the rendered PNG (with automatic token compaction and 0.7x LANCZOS image downscaling on context overflow).
3. **`judge_readability(run)`**:
   - **Pixel-border clipping detector**: Estimates the canvas background color from the four corners and measures non-background ink density along the outer 2-pixel borders (`top`, `bottom`, `left`, `right`) to catch titles or tick labels clipped by the image boundary.
   - **Resolution, aspect ratio, and subplot collision checks**: Flags figures rendered below `320x240`, extreme aspect ratios (`> 4.5:1`), or multi-panel `subplots(2, ...)` grids rendered without `tight_layout()`, `constrained_layout`, or `subplots_adjust`.

---

2. Part 2 — Adversarial Task Authoring & Multi-Agent Evaluation

2.1 Authored Task Suite (`tasks/task_01`–`task_05`)
We authored five self-contained CSV visualization tasks spanning two `task_class` families

- **`qwen` (`Qwen2.5-Coder-3B-Instruct`)**: Fails all 5 tasks across diverse error families: draws horizontal bars (`barh`) on `task_01` (`wrong_chart`), saves to `plots/output.png` instead of `figure.png` on `task_02` (`execution_failure`), creates an overcrowded 2x2 subplot grid without `tight_layout()` or axis labels on `task_03` (`wrong_chart`, `hard_to_read`), hardcodes bar heights without reading `cache_tiers.csv` on `task_04` (`wrong_data`), and renders a tiny `200x144` clipped canvas on `task_05` (`hard_to_read`).

2.3 Validator Performance Summary
Running `python -m validator.eval --predictions output/authored_predictions.json --labels labels.json` yields:

Error Family	Precision	Recall	F1	TP	FP	FN
`execution_failure`	1.00	1.00	1.00	1	0	0
`wrong_data`	1.00	1.00	1.00	4	0	0
`wrong_chart`	1.00	1.00	1.00	3	0	0
`hard_to_read`	1.00	1.00	1.00	2	0	0
Macro Average	**1.00**	**1.00**	**1.00**	**10**	**0**	**0**

---

3. Part 3 — Harbor Packaging & Verifier Gap Analysis

3.1 Harbor Environment Design (`harbor/tasks/task_01`–`task_05`)
Each authored task is packaged as a standalone Harbor benchmark directory passing `python harbor/preflight.py` with 0 errors and 0 warnings:
1. **Normative Sidecar & Re-Executable Script Contract (`instruction.md`)**: Because raw PNG pixels do not encode exact floating-point bar heights or prove that the numbers came from the CSV rather than hand-typed constants, `instruction.md` requires three deliverables in `/app`:
   - `/app/plot.py` (self-contained, deterministic script using relative paths),
   - `/app/figure.png` (the chart saved by `plot.py`), and
   - `/app/plotted_values.json` (a JSON object mapping each category key to its exact numeric value).
2. **Fail-Closed Verifier (`tests/test_state.py`)**:
   - `test_figure_exists` & `test_figure_is_a_valid_nonblank_png`: Rejects missing, truncated (`<= 1000` bytes), corrupt, or blank (`<= 4` unique RGB colors) images (`nop` agent scores `0.0`).
   - `test_input_csv_was_not_modified`: Verifies the exact SHA-256 digest of the input CSV so an agent cannot cheat by editing the input file.
   - `test_sidecar_parses_and_matches_expected_values`: Enforces exact category keys, JSON number types (rejecting strings and booleans), and `pytest.approx(EXPECTED, rel=1e-5)`.
   - `test_script_reruns_and_tracks_perturbed_input`: Copies `plot.py` into an isolated temporary directory, appends a synthetic perturbation row (`EXTRA_ROW`) to the CSV, re-executes `python3 plot.py`, and asserts that the newly emitted `plotted_values.json` shifts to `PERTURBED`. This defeats hardcoded answers (`mutants/hardcoded`).

3.2 Mutant Study: Measured Verifier Coverage vs. Blind Spots
In each `harbor/tasks/<task_id>/mutants/` directory, we provide two plausible-but-wrong mutant solutions:

1. **Rejected Mutant (`mutants/unfiltered_rows/solve.sh` → Reward = `0.0`)**:
   - **What it does**: Reads the CSV, computes the derived metric, writes both `figure.png` and `plotted_values.json`, and updates dynamically under input perturbation—but omits the `if row[filter_col] != filter_val: continue` filter.
   - **Why it fails**: Caught deterministically by `test_sidecar_parses_and_matches_expected_values` (`AssertionError: plotted_values.json values [40.0, 58.0, 42.0, 40.0] do not match expected [24.0, 34.0, 22.0, 30.0]`).
2. **Accepted Mutant (`mutants/unverified_styling/solve.sh` → Reward = `1.0`)**:
   - **What it does**: Computes the exact filtered aggregations from the CSV, writes a valid `plotted_values.json`, tracks the perturbed CSV row on re-execution, and saves a valid PNG with `> 4` distinct RGB colors—but sets the bar fill color to `#FEFEFE` (nearly invisible against the white `#FFFFFF` background) and sets all title, label, and tick font sizes to `1pt` (`item.set_fontsize(1)`).
   - **Why it passes (The Verifier Gap)**: Our programmatic verifier checks file existence, PNG decoding, `> 4` distinct RGB colors, CSV SHA-256 integrity, `plotted_values.json` numerical accuracy, and perturbation responsiveness—all of which `unverified_styling` satisfies! However, a human reader cannot read `1pt` text or see `#FEFEFE` bars on a `#FFFFFF` canvas. Closing this perceptual gap requires either matplotlib artist-tree introspection (via a `sitecustomize.py` `Figure.savefig` hook inspecting font sizes and RGBA contrast) or a visual readability judge (`judge_readability`).